Texas Data Repository file-level assessment script

Metadata

- *Version*: 0.0.7
- *Released*: 2025/12/15
- *Author(s)*: Bryan Gee (UT Libraries, University of Texas at Austin; bryan.gee@austin.utexas-.edu; ORCID: 0000-0003-4517-3290)
- *Contributor(s)*: None
- *License*: GNU GPLv3
- *README last updated*: 2025/12/15

Overview

This is the Jupyter Notebook version of the *dataverse-file-assessment.py* script. It contains the same functionality but is designed to provide a more detailed explanation of different steps and is targeted at an introductory level of familiarity with Python, APIs, and Dataverse. In theory, it should be accessible to a new institutional liaison for the Texas Data Repository (TDR) within a few weeks of their introduction to TDR as a whole.

Modules

You will need the following modules; several of these are not included in the default installation of Python and will need to be installed (e.g., through pip). If you need help getting started with how to install packages, please refer to this link: https://packaging.python.org/en/latest/tutorials/installing-packages/

```
import json
import os
import math
import pandas as pd
import requests
from datetime import datetime
from rapidfuzz import process, fuzz
from urllib.parse import urlparse, parse_qs
```

Toggles

Toggles are Boolean (TRUE/FALSE) variables that define whether part of the workflow should or shouldn't happen or in what fashion it should happen. For example, the *test* toggle will enable or disable 'Test' mode; when 'Test' is set to TRUE, it will cap how many records it retrieves in order to do a really quick run. Depending on the workflow and what you set the cap at, this could cut the runtime by as much as 99% to under a minute. This is ideal for making sure that the workflow as you have (re)configured it will work all the way through, rather than running in non-test mode

and getting 80% of the way into an hour-long process, only to discover you have a code-breaking error at the end. There are four toggles in this workflow:

- **test**: as noted above, this will enable or disable 'Test' mode. The restricted retrieval caps are defined in the *config.json* file (see next section).
- **onlyMyInstitution**: this toggle will define whether to look at only your institution (e.g., 'UT Austin') or all institutions in TDR. As a note, only a superuser will be able to retrieve certain records (e.g., drafts) for all dataverses; institutional liaisons will not be able to get this information for an institution that they don't belong to.
- **versionsAPI**: the primary API endpoint that retrieves detailed metadata on individual datasets/files is the Native API. This endpoint only returns the most recent version's metadata. Therefore, if metadata has changed over time, you may not get a complete record of something like total file size (if files have been replaced/deleted) or authorship (if authors have been removed) because everything except the latest version is not retrieved. The versions API will return the same level of detail for all versions, but it is more time-intensive because of this, and in some instances, it may be known or suspected that outdated versions do not contribute significantly (e.g., a dataset that has only been versioned to add metadata like keywords, without file changes). If the toggle is set to 'TRUE,' the script will loop through this endpoint as well.
- **excludeDrafts**: related to *onlyMyInstitution*, if you want to ensure standardized results across all institutions and don't have superuser permissions, you may want to toggle this on to export versions of the final outputs that omit unpublished dataset.

```
#toggle for test environment (incomplete run, faster to complete)
test = True
#toggle to only look at your/one institution in TDR
only_my_institution = True
#toggle for stage 3 retrieval
versions_API = False
#toggle for excluding unpublished
exclude_drafts = True
```

Dates

Both for filenames and in order to calculate how long the script takes, we define two timestamps. These can then be dynamically called through the script.

```
#setting timestamp at start of script to calculate run time
start_time = datetime.now()
#creating variable with current date for appending to filenames
today = datetime.now().strftime("%Y%m%d")
```

config file

The *config.json* file, which can be variably named and in various formats (e.g., Michael uses a .env text file), provides parameters for the analysis. Externalizing the parameters helps to keep the script clean, since most parameters don't need to be updated regularly, and protects sensitive in-

formation like API keys. This file should never be distributed to anyone else; instead, share the *config-template.json* file.

```
#read in config file
with open('config.json', 'r') as file:
    config = json.load(file)
```

Because you can't "comment out" remarks in a JSON file in the same way that you can with Python, I want to provide some comments on what's in the config file below. The first example is showing you what's in the 'INSTITUTION' block, which is various parameters for a specific institution. You will want to modify these in the same syntax for your own institution.

```
my_institution = config['INSTITUTION']
print(my_institution)
```

```
{'name': 'University of Texas at Austin', 'filename': 'UT-
austin', 'uniqueIdentifier': 'Austin', 'ror': 'https://ror.org/00hj54h04',
'myInstitution': 'UT Austin'}
```

You don't need to load in sequential levels of a nested object; you can go directly to the field you want, as shown below.

```
##read in filename version of your institution's name
my_institution_filename = config['INSTITUTION']['filename']
###condition what goes in the filename based on toggle for which institution(s)
to ping
if only_my_institution:
    institution_filename = my_institution_filename
else:
    institution_filename = "all-institutions"
##read in short-hand version of your institution's name
my_institution_shortName = config["INSTITUTION"]["myInstitution"]

print(f'String to add to filenames: {my_institution_filename}.\n')
print(f'Short hand version of institution name: {my_institution_shortName}.\n')
```

```
String to add to filenames: UT-austin.

Short hand version of institution name: UT Austin.
```

The 'VARIABLES' field is what dictates the size of the retrieval to be made. Limits on attributes like page size (how many in one call), page count (how many in a total call), and starting parameters will be defined in API documentation: https://guides.dataverse.org/en/latest/api/index.html. Each API is slightly different, so you will need to adapt code based on the API. This script only uses the

Dataverse API, but if you need help with the Crossref, DataCite, Dryad, Figshare, OpenAlex, or Zenodo APIs, reach out to Bryan who has code for all of these. For this specific script, you should (be able to) leave these API parameters as is. The following variable is only called here to show the entire field; the actual parameters utilized are called directly later on.

```
apiParams = config['VARIABLES']
print(apiParams)
```

```
{'PAGE_SIZES': {'datacite_prod': 1000, 'datacite_test': 200, 'dataverse':
200}, 'PAGE_LIMITS': {'datacite_test': 5, 'datacite_prod': 600, 'dspace_test':
3, 'dspace_prod': 100, 'tdr_test': 10, 'tdr_prod': 100}, 'PAGE_STARTS':
{'datacite': 0, 'dataverse': 0}, 'PAGE_INCREMENTS': {'dataverse': 1}}
```

Not all of the fields (either first-order or nested fields) are used for this script. Some of them pertain only to the other script in this repository (which isn't really that related to be honest). You won't need to bother with the various permutations of your institution's name, for example. Most of the config file is taken up by these dictionaries of key-value pairs or lists of strings. For example, the 'WORDS' field shown below is a series of words that are considered non-descriptive; these are used for a metadata assessment of dataset titles down the line.

```
words = config['WORDS']
print(words)
```

```
{'articles': ['a', 'the', 'thee', 'an'], 'conjunctions': ['and', 'or',
'but'], 'prepositions': ['in', 'to', 'of', 'for', 'with', 'as', 'from'],
'auxiliary_verbs': ['is', 'are', 'was', 'were'], 'possessives': ['our',
'my'], 'descriptors': ['data', 'dataset', 'dataverse', 'repository', 'code',
'codes', 'script', 'scripts', 'software', 'supplemental', 'supplementary',
'supporting', 'figure', 'table', 'figures', 'tables', 'file', 'files', 'et al.',
'raw', 'manuscript', 'article', 'journal', 'preprint', 'replication', 'readme',
'github', 'final', 'output'], 'order': ['S1', 'S2', 'S3'], 'version': ['v1',
'v1.0', 'v2', 'v2.0', 'v3', 'v3.0']}
```

Several fields are key-value pairs of mimeType file formats and friendly file formats. These are used to systematically assess datasets and files to look for things like whether there is a code file or a README file.

```
compressed = config['COMPRESSED_FORMATS']
print(compressed)
```

```
{'application/gzip': 'GZIP (compressed archive)', 'application/x-7z-
compressed': '7z (compressed archive)', 'application/x-gzip': 'GZIP (compressed
archive)', 'application/x-rar': 'RAR (compressed archive)', 'application/x-tar':
```

```
'TAR (compressed archive)', 'application/zip': 'ZIP (compressed archive)', 'TAR/
GNU Zip (.tar.gz/.tgz) (application/x-gzip)': 'TAR.GZ (compressed archive)',
'ZIP: PKZIP (application/zip)': 'ZIP (compressed archive)'}
```

Can you just tell me what I need to do with the *config.json* file?

There are only three things you need to change in this file to tailor it for another institution:

- **Add your Dataverse API token**: this is the first field in the config file and should be blank (“”) in the template. In order to get your token, log into TDR, go to your name at the top right, and click the dropdown arrow - you should see an API token button. Do not share your token!
- **Update [‘INSTITUTION’][‘filename’]**: this is the string that will automatically be baked into filenames (so that you don’t have to manually go through and change many lines of code). This can be whatever you want.
- **Update [“INSTITUTION”][“myInstitution”]**: this is a controlled vocabulary that I cooked up for this workflow. Pick your institution out of the following:
  - “Baylor”
  - “Houston”
  - “HSC Fort Worth”
  - “SMU”
  - “TAMU”
  - “TAMU Galveston”
  - “TAMU International”
  - “Texas State”
  - “Texas Tech”
  - “Texas Women’s University”
  - “UT Arlington”
  - “UT Austin”
  - “UT San Antonio Health”
  - “UT Southwestern Medical”

Specifically for Texas A&M, enter “TAMU” to get records from all three TAMU campuses; there is a downstream process to make sure that it pulls from all three dataverses. This process currently includes some institutions that are no longer members of TDR.

Remaining set-up

The next few blocks handle the rest of the the setting up of the scripting process. The first step is to create some directories. This script is pretty simple, but in order to ensure that files are consistently saved and read from the same places for everyone, the script will first look for certain subdirectories in the folder where this script is contained and make them if they don’t exist. As you can see, if the ‘Test’ toggle is enabled, it will make a *test* subdirectory and change the directory to *test*. So test and non-test outputs are separated to avoid any confusion.

```
#getting script directory
script_directory = os.getcwd()
print(f"The script directory is {script_directory}.\n")

#creating directories
if test:
    if os.path.isdir("test"):
        print("test directory found - no need to recreate")
    else:
        os.mkdir("test")
        print("test directory has been created")
    os.chdir('test')
    if os.path.isdir("outputs"):
        print("test outputs directory found - no need to recreate")
    else:
        os.mkdir("outputs")
        print("test outputs directory has been created")
else:
    if os.path.isdir("outputs"):
        print("outputs directory found - no need to recreate")
    else:
        os.mkdir("outputs")
        print("outputs directory has been created")
```

```
The script directory is c:\Users\bmg3525\OneDrive - The University of Texas at
Austin\Documents\scripts\tdr-assessment-scripts.

test directory found - no need to recreate
test outputs directory found - no need to recreate
```

Utilized API endpoints

For maximum coverage, we need to query three different endpoints in the Dataverse API. A comparison of what information is retrieved from these

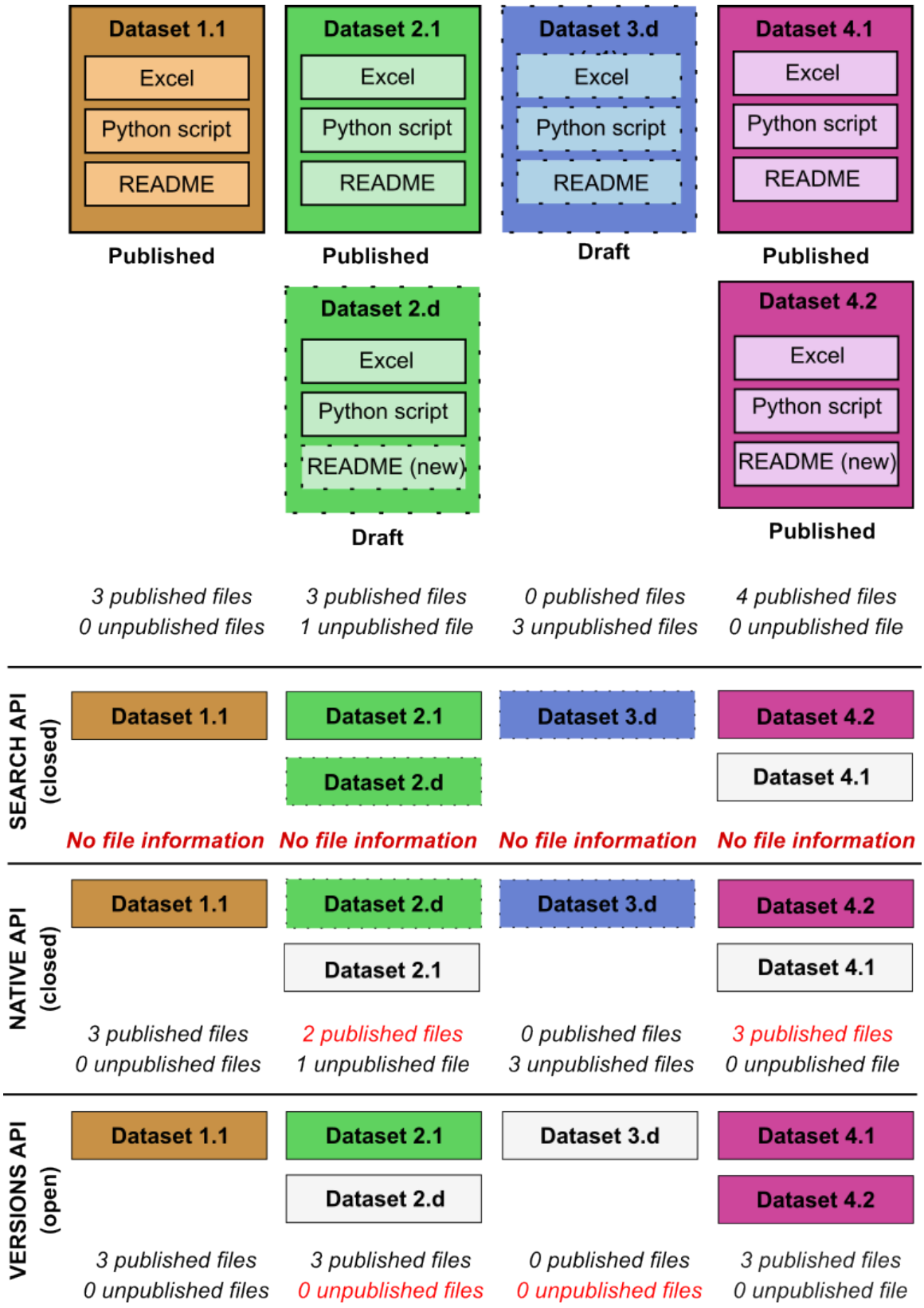

Figure 1: Comparison of information retrieved from Dataverse endpoints

Basic API params

The next step is basic API parameters. This block defines the first API endpoint (the Search API). It then dynamically calls in certain parameters about retrieval size limits from the *config.json* file, with different values based on whether you are in the Test mode. The API key is also called in. The value '*k*' is kept in here, instead of the *config.json* file, because it should always be set as zero (i.e. don't touch that). It's used to count pages while the API call is being made. The *query* value is set as the DOI prefix for all TDR deposits - this is because you cannot do a blank search through the API, so you need a value that you know will return 100% of datasets.

```
print("Beginning to define API call parameters.")
url_tdr = "https://dataverse.tdl.org/api/search/"

##set API-specific params
###Dataverse
page_limit_dataverse = config['VARIABLES']['PAGE_LIMITS']['tdr_test'] if test
else config['VARIABLES']['PAGE_LIMITS']['tdr_prod']
page_size  =  config['VARIABLES']['PAGE_SIZES']['dataverse']  if  test  else
config['VARIABLES']['PAGE_SIZES']['dataverse']

print(f"Retrieving {page_size} records per page over {page_limit_dataverse}
pages.")

###for TDR, affiliation is not reliable for returning all relevant results; the
DOI prefix is used as the most generic common denominator for datasets
query = '10.18738/T8/'
page_start_dataverse = config['VARIABLES']['PAGE_STARTS']['dataverse']
page_increment = config['VARIABLES']['PAGE_INCREMENTS']['dataverse']
k = 0

headers_tdr = {
    'X-Dataverse-key': config['KEYS']['dataverseToken']
}
```

```
Beginning to define API call parameters.
Retrieving 200 records per page over 10 pages.
```

Institution-specific parameters

The next codeblock is using what's called the 'subtree'; this is a field that contains a unique code for each institution and directs to your institution's specific dataverse. Each institution has its own set of parameters, which are largely constant between them except for the subtree, defined.

```
params_tdr_ut_austin = {
    'q': query,
    'subtree': 'utexas',
    'type': 'dataset',
```

```
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_baylor = {
    'q': query,
    'subtree': 'baylor',
    'type': 'dataset',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_smu = {
    'q': query,
    'subtree': 'smu',
    'type': 'dataset',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_tamu = {
    'q': query,
    'subtree': 'tamu',
    'type': 'dataset',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_txst = {
    'q': query,
    'subtree': 'txst',
    'type': 'dataset',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_ttu = {
    'q': query,
    'subtree': 'ttu',
    'type': 'dataset',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
```

```
}

params_tdr_houston = {
    'q': query,
    'subtree': 'uh',
    'type': 'dataset',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_hscfw = {
    'q': query,
    'subtree': 'unthsc',
    'type': 'dataset',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_tamug = {
    'q': query,
    'subtree': 'tamug',
    'type': 'dataset',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}
params_tdr_tamui = {
    'q': query,
    'subtree': 'tamiu',
    'type': 'dataset',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}
params_tdr_utsah = {
    'q': query,
    'subtree': 'uthscsa',
    'type': 'dataset',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}
params_tdr_utswm = {
    'q': query,
    'subtree': 'utswmed',
    'type': 'dataset',
```

```
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_uta = {
    'q': query,
    'subtree': 'uta',
    'type': 'dataset',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_twu = {
    'q': query,
    'subtree': 'twu',
    'type': 'dataset',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}
```

Parameter combinations

The next codeblock is setting up the code to handle different conditions based on whether you want to look at only your institution or all TDR institutions. The *all_params* object combines all of the institution-specific parameters. The *tamu_combined_params* object combines only TAMU parameters. In theory, you can cook up any combination you want (e.g., UT system schools).

```
all_params = {
        "UT Austin": params_tdr_ut_austin,
        "Baylor": params_tdr_baylor,
        "SMU": params_tdr_smu,
        "TAMU": params_tdr_tamu,
        "Texas State": params_tdr_txst,
        "Texas Tech": params_tdr_ttu,
        "Houston": params_tdr_houston,
        "HSC Fort Worth": params_tdr_hscfw,
        "TAMU Galveston": params_tdr_tamug,
        "TAMU International": params_tdr_tamui,
        "UT San Antonio Health": params_tdr_utsah,
        "UT Southwestern Medical": params_tdr_utswm,
        "UT Arlington": params_tdr_uta,
        "Texas Women's University": params_tdr_twu
    }

tamu_combined_params = {
```

```
        "TAMU": params_tdr_tamu,
        "TAMU Galveston": params_tdr_tamug,
        "TAMU International": params_tdr_tamui
}
```

The next codebook then uses 'if-else' statements to tell the script which combination (or single-institution parameter set) to use. The script is basically saying:

- if you set the *onlyMyInstitution* toggle to TRUE, AND you set the *my_institution_shortName* to "TAMU," then I will use the combined TAMU parameter set (*tamu_combined_params*).
- if you set the *onlyMyInstitution* toggle to TRUE, BUT you set the *my_institution_shortName* to anything other than "TAMU," then I will use the single institution parameter set (e.g., *params_tdr_ut_austin* for UT Austin).
- if you set the *onlyMyInstitution* toggle to FALSE, then I will search across all institutions with *all_params*.

```
#substitute for your institution
if only_my_institution:
    if my_institution_shortName == "TAMU":
        params_list = tamu_combined_params
    else:
        params_list = {
            my_institution_shortName: all_params[my_institution_shortName]
        }
else:
    params_list = all_params
```

Functions

Functions are used when you need to apply the same logic many times throughout a script; it can make the process more concise rather than having to repeat tens to hundreds of lines of code. To be honest, the functions below are pretty technical and are the result of a lot of experimentation with many APIs. You should not touch them unless you know what you are doing, but feel free to ask questions if you want to know what a given line is doing. As a note, even though some functions' names might suggest they are designed only for querying all institutions (e.g., *retrieve_all_data_for_institutions*), if you are only running this for a single institution or a subset of institutions, it will still work the same.

```
#define functions
##function to get single page from Dataverse API
def retrieve_page_dataverse(url, params=None, headers=None):
    try:
        response = requests.get(url, params=params, headers=headers)
        response.raise_for_status()
        return response.json()
    except requests.RequestException as e:
```

```
        print(f"Error retrieving page: {e}")
        return {'data': {'items': [], 'total_count': 0}}
##function to get many pages from Dataverse API
def retrieve_all_data_dataverse(url, params, headers):
    global page_limit_dataverse, k
    # create empty list
    all_data_tdr = []

    while True and k < page_limit_dataverse:
        k+=1
        data = retrieve_page_dataverse(url, params, headers)
        total_count = data['data']['total_count']
        total_pages = math.ceil(total_count/page_limit_dataverse)
        print(f"Retrieving Page {params['page']} of {total_pages} pages...\n")

        if not data['data']:
            print("No data found.")
            break

        all_data_tdr.extend(data['data']['items'])

        #update pagination
        params['start'] += page_limit_dataverse
        params['page'] += 1

        if params['start'] >= total_count:
            print("End of response.")
            break

    return all_data_tdr

##function to retrieve many pages from many institutions
def retrieve_all_data_for_institutions(url, params_list, headers):
    all_data = []

    for institution_name, params in params_list.items():
        global page_limit_dataverse, k
        k = 0  #reset k for each institution

        all_data_tdr = retrieve_all_data_dataverse(url, params, headers)
        for entry in all_data_tdr:
            entry['institution'] = institution_name
            all_data.append(entry)

    return all_data


##function to count descriptive words
```

```
def count_words(text):
    words = text.split()
    total_words = len(words)
        descriptive_count = sum(1 for word in words if word not in
nondescriptive_words)
    return total_words, descriptive_count

##function to account for when a single word may or may not be descriptive but
is certainly uninformative if in a certain combination
def adjust_descriptive_count(row):
    if ('supplemental material' in row['title_reformatted'].lower() or
            'supplementary material' in row['title_reformatted'].lower() or
            'supplementary materials' in row['title_reformatted'].lower() or
            'supplemental materials' in row['title_reformatted'].lower()):
        return max(0, row['descriptive_word_count_title'] - 1)
    return row['descriptive_word_count_title']

##function to assign size bins (file or dataset level)
def assign_size_bins(df, column='file_size', new_column='file_size_bin'):
    df=df.copy()
    bins = [
        (0, 1 * 1024, "0-10 kB"),
        (1 * 1024, 1 * 1024 * 1024, "10 kB-1 MB"),
        (1 * 1024 * 1024, 100 * 1024 * 1024, "1-100 MB"),
        (100 * 1024 * 1024, 1 * 1024 * 1024 * 1024, "100 MB-1 GB"),
        (1 * 1024 * 1024 * 1024, 10 * 1024 * 1024 * 1024, "1-10 GB"),
        (10 * 1024 * 1024 * 1024, 15 * 1024 * 1024 * 1024, "10-15 GB"),
        (15 * 1024 * 1024 * 1024, 20 * 1024 * 1024 * 1024, "15-20 GB"),
        (20 * 1024 * 1024 * 1024, 25 * 1024 * 1024 * 1024, "20-25 GB"),
        (25 * 1024 * 1024 * 1024, 30 * 1024 * 1024 * 1024, "25-30 GB"),
        (30 * 1024 * 1024 * 1024, 40 * 1024 * 1024 * 1024, "30-40 GB"),
        (40 * 1024 * 1024 * 1024, 50 * 1024 * 1024 * 1024, "40-50 GB"),
    ]

    #default set to empty
    df[new_column] = "Empty"
    for lower, upper, label in bins:
       df.loc[(df[column] > lower) & (df[column] <= upper), new_column] = label
    #maximum bin
    df.loc[df[column] > 50 * 1024 * 1024 * 1024, new_column] = ">50 GB"

    return df

##function to return only the highest value for the version number
def extract_max_version(val):
    if isinstance(val, str):
        try:
            versions = [float(v.strip()) for v in val.split(';')]
```

```
            return max(versions)
        except ValueError:
            return val  # In case of unexpected format
    return val
```

(Optional): Pull in database of ROR-matched affiliations

If you are seeking to use this process to batch re-curate existing datasets that are deficient for ROR IDs, you will want to create a 'master' file that you continue to build on where ROR IDs are manually matched for all entered affiliations (or as many as possible) in your TDR instance.

```
file_path = f'{script_directory}/tdr-affiliation-ror-matching.csv'

if os.path.exists(file_path):
    master_ror_matching = pd.read_csv(file_path)
    print(f"'{file_path}' exists and has been loaded into a DataFrame.")
else:
    print(f"'{file_path}' does not exist. DataFrame not loaded.")
```

```
'c:\Users\bmg3525\OneDrive - The University of Texas at Austin\Documents\scripts\tdr-assessment-scripts/tdr-affiliation-ror-matching.csv' exists and has been loaded into a DataFrame.
```

Retrieval process: part 1

Because we defined a series of functions related to retrieving records from the Dataverse API, potentially across all institutions, the process to trigger the API call is a single line, as seen below.

```
print("Starting TDR retrieval.\n")
all_data = retrieve_all_data_for_institutions(url_tdr, params_list, headers_tdr)
```

```
Starting TDR retrieval.

Retrieving Page 1 of 171 pages...

Retrieving Page 2 of 171 pages...

Retrieving Page 3 of 171 pages...

Retrieving Page 4 of 171 pages...

Retrieving Page 5 of 171 pages...

Retrieving Page 6 of 171 pages...
```

```
Retrieving Page 7 of 171 pages...

Retrieving Page 8 of 171 pages...

Retrieving Page 9 of 171 pages...

Retrieving Page 10 of 171 pages...
```

Subsetting API response

After the call, the API response is cached in the system but not saved in any form. You could, if you wanted to, export it as a JSON file, but I prefer working in CSV / tabular format for a lot of this work, so we'll convert it to a dataframe first. The first step in that is selecting certain fields that we want; not all fields are useful.

Why not just explode each JSON field into a separate Excel column (e.g., *authors* becomes *authors_1, authors_2,* etc.)?

I actually used to do this when I was developing another Python workflow (https://github.com/utlibraries/research-data-discovery). What I discovered is that, given any appreciable number of dataset records (let's say more than 250), at least one is virtually guaranteed to have an inordinate number of nested fields (e.g., 29 authors). This means that the script will flatten just that one field into 29 columns, which takes a lot longer to do and a lot longer to work through. If you start getting up above 1,000 records, you might not have enough RAM to actually do this. When we know what fields are important, we can just subset for them.

Why are we subsetting so few fields?

The Search API endpoint returns relatively little metadata because it's intended for a broad capture (one call, many results). It doesn't even return information like author affiliations. So the main purpose of this call is just to get every single dataset of interest, and then we'll pass the DOIs into a different API endpoint to get more detailed metadata.

The way that this first subsetting process works is to create an empty list to hold the subsetted output (*data_select_tdr*) and then to loop through the full API response (*all_data*), pulling out specific fields. The syntax (e.g., *item.get*) is based on the hierarchy of the output. You'll see in later processes how we'll need to extract more nested objects. The script then appends each dataset's subsetted metadata into the list, and we convert it to a dataframe with the *pandas* library.

```
print("Starting TDR filtering.\n")
data_select_tdr = []
for item in all_data:
    id = item.get('global_id', '')
    type = item.get('type', '')
    institution = item.get('institution','')
    status = item.get('versionState', '')
    name = item.get('name', '')
```

```
    dataverse = item.get('name_of_dataverse', '')
    majorV = item.get('majorVersion', 0)
    minorV = item.get('minor_version', 0)
    comboV = f"{majorV}.{minorV}"
    version_id = item.get('versionId', '')
    data_select_tdr.append({
        'institution': institution,
        'doi': id,
        'type': type,
        'status': status,
        'dataset_title': name,
        'dataverse': dataverse,
        'major_version': majorV,
        'minor_version': minorV,
        'total_version': comboV,
        'version_id': version_id
    })

df_data_select_tdr = pd.DataFrame(data_select_tdr)
```

```
Starting TDR filtering.
```

Post-conversion cleaning

We'll do a little bit of cleaning / modification of the data now that they're in a dataframe. The first step (filtering for 'type' = 'dataset') is redundant now because the 'type' is specified in the API call (this omits retrieval of 'dataverse' and 'file' records), but it's retained because it doesn't take up any more time and can be useful if you change the parameters later. The two other steps edit the DOI field to remove a 'doi:' string that always precedes the DOI value in that column and to add a Boolean column indicating whether a dataset has been versioned or not.

```
#remove dataverses and files
filtered_tdr = df_data_select_tdr[df_data_select_tdr['type'] == 'dataset']
#editing DOI field
filtered_tdr['doi'] = filtered_tdr['doi'].str.replace('doi:', '')
#add column for versioned
filtered_tdr['versioned']  =  filtered_tdr.apply(lambda  row:  'Versioned'  if
(row['major_version'] > 1) or (row['minor_version'] > 0) else 'Not versioned',
axis=1)
```

Metadata assessment: part 1

We can't really do too much metadata assessment at the dataset/file level because of the limited metadata we get from the Search API. We can do one though: how descriptive are dataset titles? This pulls in a list of nondescriptive words from the *config.json* file (you can modify these if you wish) and then looks through the title to count how many other words remain. This operates on a few combinations of words as well (*adjust_descriptive_count* function) where the individual words

aren't necessarily nondescriptive in isolation but are when combined (e.g., 'supplemental information'). Titles are formatted to remove underscores and hyphens, otherwise the entire string gets treated as one word.

```
#metadata assessments
##title
##assess 'descriptiveness of dataset title'
words = config['WORDS']
###add integers
numbers = list(map(str, range(1, 1000000)))
###combine all into a single set
nondescriptive_words = set(
    words['articles'] +
    words['conjunctions'] +
    words['prepositions'] +
    words['auxiliary_verbs'] +
    words['possessives'] +
    words['descriptors'] +
    words['order'] +
    words['version'] +
    numbers
)

filtered_tdr['title_reformatted']                                          =
filtered_tdr['dataset_title'].str.replace('_', ' ')
filtered_tdr['title_reformatted']                                          =
filtered_tdr['dataset_title'].str.replace('-', ' ') #gets around text linked by
underscores counting as 1 word
filtered_tdr['title_reformatted']                                          =
filtered_tdr['title_reformatted'].str.lower()
filtered_tdr[['total_word_count_title',    'descriptive_word_count_title']]   =
filtered_tdr['title_reformatted'].apply(lambda x: pd.Series(count_words(x)))

filtered_tdr['descriptive_word_count_title']                               =
filtered_tdr.apply(adjust_descriptive_count, axis=1)
filtered_tdr['nondescriptive_word_count_title']
=                    filtered_tdr['total_word_count_title']                -
filtered_tdr['descriptive_word_count_title']
```

Retrieval process: part 2

The next part of the process is where we get richer metadata. In this step, we go through the Native API. This endpoint requires you to feed a single DOI into it, and then it returns very detailed, file-level metadata for the latest version of the dataset (regardless of publication status).

First, however, we need to deal with a quirk of the Search API: that it returns two records for any dataset that has been previously published and that is now in draft status. These records could be functionally identical (e.g., author accidentally puts published dataset into draft mode, strands

it there without making changes) or quite significantly different. The code below de-duplicates these records, retaining only the published version, but saves a CSV file with a list of pairs of records with this status.

```
#sort on status, setting 'DRAFT' at bottom to remove this version for published datasets that are in draft state, retain entry of 'PUBLISHED'
filtered_tdr = filtered_tdr.sort_values(by='status', ascending=False)
filtered_tdr.to_csv(f"outputs/{today}_{institution_filename}_all-deposits.csv")
filtered_tdr_deduplicated    =    filtered_tdr.drop_duplicates(subset=['doi'], keep="first")
filtered_tdr_deduplicated.to_csv(f"outputs/{today}_{institution_filename}_all-deposits-deduplicated.csv")
print(f'Total datasets to be analyzed: {len(filtered_tdr_deduplicated)}.\n')

#create df of published datasets with draft version (retains both entries)
commonColumns = ['doi', 'dataset_title']
duplicates = filtered_tdr.duplicated(subset=commonColumns, keep=False)
dualStatusDatasets = filtered_tdr[duplicates]
dualStatusDatasets.to_csv(f"outputs/{today}_{institution_filename}_dual-status-datasets.csv")
```

```
Total datasets to be analyzed: 100.
```

Now we have a list of unique DOIs that we want more metadata on. The following code makes a similar API call to the first one, but through the Native API endpoint. So we define the endpoint (it's a different URL) and then set up a 'for loop,' where we go through each DOI in the de-duplicated dataframe, get its metadata, and save that in an empty list called *results*.

```
#retrieving additional metadata for deposits by individual API call (one per DOI)
##retrieves both published and never-published draft datasets; if a published dataset is currently in DRAFT state, it will return the information for the DRAFT state
print("Starting Native API call")
url_tdr_native = "https://dataverse.tdl.org/api/datasets/"

results = []
for doi in filtered_tdr_deduplicated['doi']:
    try:
                    response  =  requests.get(f'{url_tdr_native}:persistentId/?persistentId=doi:{doi}', headers=headers_tdr, timeout=5)
        if response.status_code == 200:
            print(f"Retrieving {doi}\n")
            results.append(response.json())
```

```
        else:
                    print(f"Error retrieving {doi}: {response.status_code},
{response.text}")
    except requests.exceptions.RequestException as e:
        print(f"Timeout error on DOI {doi}: {e}")

data_tdr_native = {
    'datasets': results
}
```

```
Starting Native API call
Retrieving 10.18738/T8/RUFGR9

Retrieving 10.18738/T8/OUDSWK

Retrieving 10.18738/T8/39LEFA

Retrieving 10.18738/T8/CGCOU2

Retrieving 10.18738/T8/UC7SLS

Retrieving 10.18738/T8/UKAD3W

Retrieving 10.18738/T8/4EZK6T

Retrieving 10.18738/T8/CMGYKD

Retrieving 10.18738/T8/NINUKS

Retrieving 10.18738/T8/XFPTF7

Retrieving 10.18738/T8/97ZAZN

Retrieving 10.18738/T8/AEOGII

Retrieving 10.18738/T8/MA0GRX

Retrieving 10.18738/T8/SR6VQP

Retrieving 10.18738/T8/HNHTOD

Retrieving 10.18738/T8/PEAPF6

Retrieving 10.18738/T8/UO4F9F

Retrieving 10.18738/T8/J38CO5
```

```
Retrieving 10.18738/T8/ZQLXF3

Retrieving 10.18738/T8/A5UK5R

Retrieving 10.18738/T8/ZEDKNG

Retrieving 10.18738/T8/AMRCQM

Retrieving 10.18738/T8/XB6EMO

Retrieving 10.18738/T8/Z3GDUH

Retrieving 10.18738/T8/RHRMQ7

Retrieving 10.18738/T8/NBVNH6

Retrieving 10.18738/T8/IML5PB

Retrieving 10.18738/T8/GHRTZQ

Retrieving 10.18738/T8/47H1UK

Retrieving 10.18738/T8/MCHHL5

Retrieving 10.18738/T8/AHZJYY

Retrieving 10.18738/T8/BJSMGU

Retrieving 10.18738/T8/Y9HKXG

Retrieving 10.18738/T8/42R37Y

Retrieving 10.18738/T8/ZLQOBS

Retrieving 10.18738/T8/NHX2VA

Retrieving 10.18738/T8/3DDN6Z

Retrieving 10.18738/T8/QN0A2C

Retrieving 10.18738/T8/ACYGIN

Retrieving 10.18738/T8/26DDUF

Retrieving 10.18738/T8/FVJYLS

Retrieving 10.18738/T8/BWQA2P
```

```
Retrieving 10.18738/T8/UT3MME

Retrieving 10.18738/T8/R7XZMJ

Retrieving 10.18738/T8/HNW7TJ

Retrieving 10.18738/T8/0SB4VL

Retrieving 10.18738/T8/A3I0ZY

Retrieving 10.18738/T8/ZPRB4W

Retrieving 10.18738/T8/FITVEP

Retrieving 10.18738/T8/ZIMOMB

Retrieving 10.18738/T8/KBZWKN

Retrieving 10.18738/T8/OQXGVP

Retrieving 10.18738/T8/0NXOPR

Retrieving 10.18738/T8/7BDOXR

Retrieving 10.18738/T8/0PRYRH

Retrieving 10.18738/T8/C8IE7C

Retrieving 10.18738/T8/CQHJPL

Retrieving 10.18738/T8/BXJBF5

Retrieving 10.18738/T8/PNQCJ4

Retrieving 10.18738/T8/AKBROP

Retrieving 10.18738/T8/3VUPEW

Retrieving 10.18738/T8/5MPOJU

Retrieving 10.18738/T8/700QAK

Retrieving 10.18738/T8/YIGGX7

Retrieving 10.18738/T8/NMQA1N

Retrieving 10.18738/T8/HJWR9E
```

```
Retrieving 10.18738/T8/MSQ7BY

Retrieving 10.18738/T8/1JSIFM

Retrieving 10.18738/T8/NVSYJC

Retrieving 10.18738/T8/YFPM85

Retrieving 10.18738/T8/KIONWW

Retrieving 10.18738/T8/DIXPTD

Retrieving 10.18738/T8/XRN25Z

Retrieving 10.18738/T8/GKHRBS

Retrieving 10.18738/T8/ZJVQCT

Retrieving 10.18738/T8/DOPVBK

Retrieving 10.18738/T8/WXH8YK

Retrieving 10.18738/T8/MADU1S

Retrieving 10.18738/T8/2QFLFG

Retrieving 10.18738/T8/UJ8TI0

Retrieving 10.18738/T8/FKLDWC

Retrieving 10.18738/T8/JKRFKT

Retrieving 10.18738/T8/ZKVBCM

Retrieving 10.18738/T8/FQAMNQ

Retrieving 10.18738/T8/L7ZTYG

Retrieving 10.18738/T8/WTQXR1

Retrieving 10.18738/T8/TBVUTG

Retrieving 10.18738/T8/KLTSXZ

Retrieving 10.18738/T8/2OK8A6

Retrieving 10.18738/T8/RA5TXQ
```

```
Retrieving 10.18738/T8/WQMHTQ

Retrieving 10.18738/T8/YUXNBP

Retrieving 10.18738/T8/G2IYLN

Retrieving 10.18738/T8/Q1ITKB

Retrieving 10.18738/T8/XYUKRD

Retrieving 10.18738/T8/IVACGI

Retrieving 10.18738/T8/6YSL5R

Retrieving 10.18738/T8/QFYJ20

Retrieving 10.18738/T8/Z3J5L8

Retrieving 10.18738/T8/OJOAPA
```

Then we do a similar subsetting process to what we did for the first API call. We make an empty list to store the output (*data_select_tdr_native*) and then loop through. You might notice that there are various *X.get* commands in the script below. This is based on the nesting of different fields. For example, *latest* is nested within *data*, so in order to get anything that is nested under *latest*, we need to define that one first. The only way to know how to structure this is to look at an actual API response. You'll see that we're subsetting a lot more fields and getting a lot more detail in the metadata. This code specifically creates an entry for each file listed in a dataset record, so some dataset-level metadata is duplicated across several rows.

```
print("Beginning dataframe subsetting\n")
data_select_tdr_native = []
for item in data_tdr_native['datasets']:
    data = item.get('data', '')
    dataset_id = data.get('id', '')
    pubDate = data.get('publicationDate', '')
    latest = data.get('latestVersion', {})
    status = latest.get('versionState', '')
    status2 = latest.get('latestVersionPublishingState', '')
    doi = latest.get('datasetPersistentId', '')
    updateDate = latest.get('lastUpdateTime', '')
    createDate = latest.get('createTime', '')
    releaseDate = latest.get('releaseTime', '')
    license = latest.get('license', {})
    licenseName = license.get('name', None)
    terms = latest.get('termsOfUse', None)
    usage = licenseName if licenseName is not None else terms
```

```
    files = latest.get('files', [])
    citation = latest.get('metadataBlocks', {}).get('citation', {})
    fields = citation.get('fields', [])
    grantAgencies = []
    for field in fields:
        if field['typeName'] == 'grantNumber':
            for grant in field.get('value', []):
                        grant_number_agency = grant.get('grantNumberAgency',
{}).get('value', '')
                grantAgencies.append(grant_number_agency)
        if field['typeName'] == 'subject':
            subjects = field.get('value', [])
        if field['typeName'] == 'keyword':
            keywords = []
            for keyword_dict in field.get('value', []):
              keyword_value = keyword_dict.get('keywordValue', {}).get('value',
'')
                if keyword_value:
                    keywords.append(keyword_value)
            keywords_str = ';'.join(keywords)
    total_filesize = 0
    unique_content_types = set()
    fileCount = len(files)
    for file in files:
        file_info = file['dataFile']
        unique_content_types.add(file_info['contentType'])
        file_entry = {
            'dataset_id': dataset_id,
            'doi': doi,
            #'status': status,
            'current_status': status2,
            'reuse_requirements': usage,
            #'fileCount': fileCount,
            #'unique_content_types': list(unique_content_types),
            'file_id': file_info.get('id', ''),
            'public': file_info.get('restricted', ''),
            'filename': file_info.get('filename', ''),
            'mime_type': file_info.get('contentType', ''),
            'friendly_type': file_info.get('friendlyType', ''),
            'tabular': file_info.get('tabularData', ''),
            'file_size': file_info.get('filesize', 0),
            'storage_identifier': file_info.get('storageIdentifier', ''),
            'creation_date': file_info.get('creationDate', ''),
            'publication_date': file_info.get('publicationDate', ''),
            'restricted': file.get('restricted', ''),
            'license': licenseName
        }
        data_select_tdr_native.append(file_entry)
```

```
Beginning dataframe subsetting
```

We're also going to create a different subsetted output from the same API response, this one for individual authors (you can create as many subsetted outputs as you want from a single API response). This one creates a separate entry for each author listed in a dataset record, so some dataset-level metadata is duplicated across several rows. This one has to deal with some extra complexity in how Dataverse structures entries that have ROR IDs vs. ones that don't.

```
#getting dataframe with entries for individual authors
author_entries = []
for item in data_tdr_native['datasets']:
    data = item.get('data', {})
    latest = data.get('latestVersion', {})
    doi = latest.get('datasetPersistentId', '')
    citation = latest.get('metadataBlocks', {}).get('citation', {})
    status2 = latest.get('latestVersionPublishingState', '')
    fields = citation.get('fields', [])
    for field in fields:
        if field['typeName'] == 'author':
            for author in field.get('value', []):
                name = author.get('authorName', {}).get('value', '')
                affiliation = author.get('authorAffiliation', {}).get('value',
'')
               identifier = author.get('authorIdentifier', {}).get('value', '')
                scheme = author.get('authorIdentifierScheme', {}).get('value',
'')
                        affiliation_expanded = author.get('authorAffiliation',
{}).get('expandedvalue', {}).get('termName', '')
                          identifier_expanded = author.get('authorIdentifier',
{}).get('expandedvalue', {}).get('@id', '')

                affiliationName = affiliation_expanded if affiliation_expanded
else affiliation
               affiliation_ror = affiliation if affiliation_expanded else None

                author_entry = {
                    'doi': doi,
                    'current_status': status2,
                    'author_name': name,
                    'author_affiliation': affiliationName,
                    'ror_id': affiliation_ror,
                    'author_identifier': identifier,
                    'author_identifier_expanded': identifier_expanded,
                    'author_identifier_scheme': scheme
                }
                author_entries.append(author_entry)
```

Initial cleaning

We then convert these subsetted responses to dataframes, standardize how the DOIs are listed, and add a column for the file-level output to create a column with just the year the file was created (from a full date).

```
df_select_tdr_native = pd.json_normalize(data_select_tdr_native)
df_author_entries = pd.json_normalize(author_entries)
df_select_tdr_native['doi'] = df_select_tdr_native['doi'].str.replace('doi:',
'')
df_author_entries['doi'] = df_author_entries['doi'].str.replace('doi:', '')
df_select_tdr_native['creation_date']                                    =
pd.to_datetime(df_select_tdr_native['creation_date'])
df_select_tdr_native['file_creation_year']                               =
df_select_tdr_native['creation_date'].dt.year
```

Metadata assessment: part 2

We do another metadata assessment at this point: binning files by their size (*size_bin*). This dataframe is then merged back with the dataframe from the Search API.

```
df_select_tdr_native          =          assign_size_bins(df_select_tdr_native,
column='file_size', new_column='file_size_bin')
df_select_concatenated          =          pd.merge(filtered_tdr_deduplicated,
df_select_tdr_native, on='doi', how="left")
df_select_concatenated_exist                                             =
df_select_concatenated.dropna(subset=['dataset_id']).copy()           #removes
deaccessioned

df_select_concatenated_exist['dataset_id']                               =
df_select_concatenated_exist['dataset_id'].astype(int)
# As of 2025/12/05, temporarily coding out this CSV output
#  df_select_concatenated_exist.to_csv(f"outputs/{today}_{institution_filename}
_all-datasets-deduplicated_expanded-metadata.csv")

#subset to datasets that are less than version 2.0 (no major update, no file
additions)
df_select_concatenated_exist_majorVersion                                =
df_select_concatenated_exist[df_select_concatenated_exist['major_version'] > 1]
```

Retrieval process: part 3

This final API call is optional (*versionsAPI* toggle). If you set that to FALSE, this entire codeblock will be skipped. If you run it, the process and metadata are nearly identical to that of the previous Native API call; the Versions endpoint is technically part of the Native API, but it works a little differently. Firstly, this endpoint is public, so it will not return any metadata on drafts of any form (drafts of previously published, drafts of never published). Secondly, it returns metadata on all published versions of a dataset (the default Native endpoint only returns the most recent version). Whether the information of older published versions will significantly impact the results

is a personal decision, and you may wish to experiment with it. The resultant output is subsetted and de-duplicated to handle predicted redundancy (e.g., a file present in versions 1 through 3 will have three entries), has the dataset size bin classification applied, and then aligns the columns to be the same as that of the previous outputs. This is done for both file-level output and author-level output.

```
#need to use Version endpoint to get info on published version of published
datasets that are currently in DRAFT status and all published versions of a
dataset with multiple PUBLISHED versions. This endpoint is public and does not
return any DRAFTs.
#remove datasets that have never been published (will not return any info for
this endpoint)
df_select_concatenated_exist_published = df_select_concatenated_exist_majorVersion[df_select_conca
#deduplicate on dataset_id
df_select_concatenated_exist_published_dedup                                    =
df_select_concatenated_exist_published.drop_duplicates(subset="dataset_id",
keep="first")

if versions_API:
    results_versions = []
    print("Beginning Version API query\n")
                                          for          dataset_id          in
df_select_concatenated_exist_published_dedup['dataset_id']:
        try:
            response = requests.get(f'{url_tdr_native}{dataset_id}/versions')
            if response.status_code == 200:
                print(f"Retrieving versions of dataset #{dataset_id}")
                print()
                results_versions.append(response.json())
            else:
                            print(f"Error  retrieving  dataset  #{dataset_id}:
{response.status_code}, {response.text}")
        except requests.exceptions.RequestException as e:
            print(f"Timeout error on DOI {doi}: {e}")

    data_tdr_versions = {
        'datasets': results_versions
    }
    print("Beginning dataframe subsetting\n")
    data_select_tdr_versions = []
    for dataset in data_tdr_versions['datasets']:
        data = dataset.get('data', [])
        for item in data:
            doi = item.get('datasetPersistentId', '')
            id = item.get("id", '')
            datasetid = item.get('datasetId', '')
            majorV = str(item.get('versionNumber', 0))
            minorV = str(item.get('versionMinorNumber', 0))
```

```
            status2 = latest.get('latestVersionPublishingState', '')
            comboV = f"{majorV}.{minorV}"
            status = item.get('versionState', '')
            license = item.get('license', {})
            licenseName = license.get('name', None)
            terms = item.get('termsOfUse', None)
            usage = licenseName if licenseName is not None else terms
            for file in item.get('files', []):
                fileInfo = file['dataFile']
                data_select_tdr_versions.append({
                    'doi': doi,
                    'version_id': id,
                    'dataset_id': datasetid,
                    #'major_version': majorV,
                    #'minor_version': minorV,
                    'total_version': comboV,
                    'file_id': fileInfo.get('id', ''),
                    'filename': fileInfo.get('filename', ''),
                    'mime_type': fileInfo.get('contentType', ''),
                    'friendly_type': fileInfo.get('friendlyType', ''),
                    #'status': status,
                    'current_status': status2,
                    #'tabular': fileInfo.get('tabularData', ''),
                    'file_size': fileInfo.get('filesize', ''),
                   'storage_identifier': fileInfo.get('storageIdentifier', ''),
                    #'md5': fileInfo.get('md5', ''),
                    'creation_date': fileInfo.get('creationDate', ''),
                    'publication_date': fileInfo.get('publicationDate', ''),
                    'restricted': file.get('restricted', ''),
                    'license': licenseName
                })
    #getting dataframe with entries for individual authors
    author_entries_versions = []
    for dataset in data_tdr_versions['datasets']:
        data = dataset.get('data', [])
        for item in data:
            doi = item.get('datasetPersistentId', '')
            id = item.get("id", '')
            status2 = item.get('latestVersionPublishingState', '')
            datasetid = item.get('datasetId', '')
            citation = item.get('metadataBlocks', {}).get('citation', {})
            fields = citation.get('fields', [])
            for field in fields:
                if field['typeName'] == 'author':
                    for author in field.get('value', []):
                        name = author.get('authorName', {}).get('value', '')
                                affiliation = author.get('authorAffiliation',
{}).get('value', '')
```

```
                    identifier = author.get('authorIdentifier', {}).get('value',
''')
                                    scheme = author.get('authorIdentifierScheme',
{}).get('value', '')
                            affiliation_expanded = author.get('authorAffiliation',
{}).get('expandedvalue', {}).get('termName', '')
                             identifier_expanded = author.get('authorIdentifier',
{}).get('expandedvalue', {}).get('@id', '')

                    affiliationName = affiliation_expanded if affiliation_expanded
else affiliation
                             affiliation_ror = affiliation if affiliation_expanded
else None

                            author_entry = {
                                'doi': doi,
                                'current_status': status2,
                                'author_name': name,
                                'author_affiliation': affiliationName,
                                'ror_id': affiliation_ror,
                                'author_identifier': identifier,
                                'author_identifier_expanded': identifier_expanded,
                                'author_identifier_scheme': scheme
                            }
                            author_entries_versions.append(author_entry)

    df_select_tdr_versions = pd.json_normalize(data_select_tdr_versions)
    df_author_entries_versions = pd.json_normalize(author_entries_versions)
                                        df_select_tdr_versions['doi']          =
df_select_tdr_versions['doi'].str.replace('doi:', '')
                                    df_author_entries_versions['doi']         =
df_author_entries_versions['doi'].str.replace('doi:', '')
     #removing duplicate entries for a given file that has not changed across
multiple versions
                                df_select_tdr_versions['total_version']        =
df_select_tdr_versions['total_version'].astype(float)
                                             df_select_tdr_versions           =
df_select_tdr_versions.sort_values(by='total_version')
                                            df_select_tdr_versions_deduplicated
=                  df_select_tdr_versions.drop_duplicates(subset=['dataset_id',
'storage_identifier'], keep='first')

                                    df_select_tdr_versions_deduplicated        =
assign_size_bins(df_select_tdr_versions_deduplicated,      column='file_size',
new_column='file_size_bin')

                                df_select_versions_concatenated_released       =
pd.merge(df_select_tdr_versions_deduplicated,       filtered_tdr_deduplicated,
```

```
on='doi', how="left")

   #pruning and renaming columns in the two dataframes that collectively (should)
have all of the files (from the Native and the Version endpoints)
                                                df_version_pruned           =
df_select_versions_concatenated_released[["version_id_x",         "dataset_id",
"totalVersion_x",   "filename",   "file_id",   "mime_type",   "friendly_type",
"file_size",    "storage_identifier",    "creation_date",    "publication_date",
"institution",    "doi",    "file_size_bin",    "dataset_title",    "dataverse",
"restricted", "license"]]
     df_version_pruned  =  df_version_pruned.rename(columns={'total_version_x':
'total_version',        'filename_x':        'filename',        'file_size_x':
'file_size',  'storage_identifier_x':  'storage_identifier',  'creation_date_x':
'creation_date',    'publication_date_x':'publication_date',    'version_id_x':
'version_id'})
                                    df_version_pruned['creation_year']        =
pd.to_datetime(df_version_pruned['creation_date'], format="%Y-%m-%d").dt.year
                                          df_version_pruned['publication_year']
=     pd.to_datetime(df_version_pruned['publication_date'],     format="%Y-%m-
%d").dt.year
```

The following block of code is run regardless of whether you queried the Versions endpoint or not. It standardizes some columns and creates some additional date columns.

```
df_native_pruned = df_select_concatenated_exist[["dataset_id", "dataset_title",
"version_id",   "current_status",   "total_version",   "filename",   "file_id",
"mime_type",      "friendly_type",      "file_size",      "storage_identifier",
"creation_date",  "publication_date",  "institution",  "doi",  "file_size_bin",
"dataverse", "restricted", "license"]]
df_native_pruned = df_native_pruned.copy()
df_native_pruned['creation_year']                                             =
pd.to_datetime(df_native_pruned['creation_date'],      format="%Y-%m-%dT%H:%M:
%SZ").dt.year
df_native_pruned['publication_year']                                          =
pd.to_datetime(df_native_pruned['publication_date'],           format="%Y-%m-
%d").dt.year
```

Conditional concatenation

If you ran the Versions endpoint process, you will need to combine that with the first Native API output and de-duplicate. This block of code uses an 'if-else' statement to either combine the two dataframes and then de-duplicate them (both file-level and author-level), or it just does the de-duplication process.

```
if versions_API:
      df_all_files_concat  =  pd.concat([df_version_pruned,  df_native_pruned],
ignore_index=True)
```

```
          df_all_files_concat   =   df_all_files_concat.rename(columns={'title':
'dataset_title'})

    #deduplicate
   ##create fake versionID for drafts to ensure proper sorting and deduplicating
                                    df_all_files_concat['version_id']         =
df_all_files_concat['version_id'].fillna(9999999)
                                    df_all_files_concat['version_id']         =
pd.to_numeric(df_all_files_concat['version_id'], errors='coerce')
    df_all_files_concat = df_all_files_concat.sort_values(by='version_id')
                                                 df_all_files_concat_deduplicated
=           df_all_files_concat.drop_duplicates(subset=['storage_identifier'],
keep='first')
    df_all_files_concat_deduplicated = df_all_files_concat_deduplicated.copy()
                           df_all_files_concat_deduplicated['version_id']      =
df_all_files_concat_deduplicated['version_id'].replace(9999999, None)
                  df_all_authors_concat     =     pd.concat([df_author_entries,
df_author_entries_versions], ignore_index=True)
                                                 df_all_authors_concat_deduplicated
=      df_all_authors_concat.drop_duplicates(subset=['doi',      'author_name',
'author_affiliation', 'current_status'], keep='first')
else:
    #sort on status and then total version, setting 'DRAFT' at bottom to remove
this version for published datasets that are in draft state, retain entry of
'PUBLISHED' and then to keep the earliest version
       df_native_pruned  =  df_native_pruned.sort_values(by=['current_status',
'total_version'], ascending=[False, True])
                                    df_all_files_concat_deduplicated         =
df_native_pruned.drop_duplicates(subset=['storage_identifier'], keep='first')
                                                 df_all_authors_concat_deduplicated
=       df_author_entries.drop_duplicates(subset=['doi',        'author_name',
'author_affiliation', 'current_status'], keep='first')
```

Metadata assessment: part 3

Finally we have reached the part that is probably most interesting to some. A wide range of metadata assessments are made here (regardless of whether you queried the Versions endpoint). This includes:

- Whether a file includes ‘readme’, ‘codebook’, or ‘dictionary’ (proxy for data dictionary) in the filename
- Converting mimeTypes to friendlyFormats; the Dataverse API does have an existing field for this, which is included in the subsetted outputs, but I find that it still needs work to standardize, and I had this giant key-value list anyway from a different script that uses APIs that don’t have friendlyFormat information.
- Whether a file is a software format (e.g., Python, R, C++)
- Whether a file is a compressed archive (e.g., .zip, .gzip, .tar)
- Whether a file is a Microsoft Office format

```
#metadata assessment
##documentation presence
df_all_files_concat_deduplicated.loc[:,'is_readme']                          =
df_all_files_concat_deduplicated['filename'].str.contains('readme',
case=False)
df_all_files_concat_deduplicated.loc[:,'is_codebook']                        =
df_all_files_concat_deduplicated['filename'].str.contains('codebook',
case=False)
df_all_files_concat_deduplicated.loc[:,'is_data_dictionary']                 =
df_all_files_concat_deduplicated['filename'].str.contains('dictionary',
case=False) #need to check sensitivity

##create separate friendlyFormat column
formatMap = config['FORMAT_MAP']
df_all_files_concat_deduplicated.loc[:,'friendly_format_manual']             =
df_all_files_concat_deduplicated['mime_type'].apply(
    lambda x: formatMap.get(x.strip(), x.strip()) if isinstance(x, str) and x !
= "no match found" else "no files"
)
##file formats
softwareFormats = set(config['SOFTWARE_FORMATS'].keys())
compressedFormats = set(config['COMPRESSED_FORMATS'].keys())
microsoftFormats = set(config['MICROSOFT_FORMATS'].keys())
# Assume softwareFormats is a set of friendly software format names
df_all_files_concat_deduplicated.loc[:,'is_software']                        =
df_all_files_concat_deduplicated['mime_type'].apply(
    lambda x: any(part.strip() in softwareFormats for part in x.split(';')) if
isinstance(x, str) else False
)
df_all_files_concat_deduplicated.loc[:,'is_compressed']                      =
df_all_files_concat_deduplicated['mime_type'].apply(
    lambda x: any(part.strip() in compressedFormats for part in x.split(';'))
if isinstance(x, str) else False
)
df_all_files_concat_deduplicated.loc[:,'is_microsoft_office']                =
df_all_files_concat_deduplicated['mime_type'].apply(
    lambda x: any(part.strip() in microsoftFormats for part in x.split(';')) if
isinstance(x, str) else False
)

# Manual file extension grabbing
df_all_files_concat_deduplicated['extension_minimum']                        =
df_all_files_concat_deduplicated['filename'].str.extract(r'(\.[^.]+)$')
df_all_files_concat_deduplicated['extension_maximum']                        =
df_all_files_concat_deduplicated['filename'].str.extract(r'(\..*)')

if exclude_drafts:
   df_all_files_concat_deduplicated_published = df_all_files_concat_deduplicated[df_all_files_conc
```

```
& (df_all_files_concat_deduplicated['publication_date'] != '')]
            df_all_files_concat_deduplicated_published.to_csv(f"outputs/{today}
_{institution_filename}_all-files-deduplicated-PUBLISHED.csv")
else:
                      df_all_files_concat_deduplicated.to_csv(f"outputs/{today}
_{institution_filename}_all-files-deduplicated-ALL.csv")
```

Now you might want to combine all of the files for a given dataset into a single entry; this would be important for things like analyzing total deposit size (inclusive of all versions) or otherwise doing some of the metadata assessments at the dataset level. The following code block does that. It first defines the column(s) to be mathematically added together (only one in this case); all others will be combined into a semi-colon-delimited string. There is also some metadata editing to clean up entries (e.g., when a recombined dataset record contains 'False; True' for a Boolean variable, this is converted to 'TRUE'). For variables where some values could be duplicated (e.g., 100 text files, only a set [unique values] is returned)

```
sum_columns = ['file_size']

def agg_func(column_name):
    if column_name in sum_columns:
        return 'sum'
    else:
        return lambda x: sorted(set(map(str, x)))

agg_funcs         =         {col:         agg_func(col)for         col         in
df_all_files_concat_deduplicated.columns if col != 'dataset_id'}

df_tdr_all_files_combined = df_all_files_concat_deduplicated.groupby('dataset_id').agg(agg_funcs).
# Convert all list-type columns to comma-separated strings
for col in df_tdr_all_files_combined.columns:
   if df_tdr_all_files_combined[col].apply(lambda x: isinstance(x, list)).any():
                                          df_tdr_all_files_combined[col]      =
df_tdr_all_files_combined[col].apply(lambda x: '; '.join(map(str, x)))

tdr_all_datasets_deduplicated                                                 =
df_tdr_all_files_combined.drop_duplicates(subset='dataset_id', keep='first')
tdr_all_datasets_deduplicated_pruned                                          =
tdr_all_datasets_deduplicated[["dataset_id",   "version_id",   "total_version",
"mime_type", "friendly_type", "file_size", "creation_date", "publication_date",
"institution",    "doi",    "dataset_title",    "dataverse",    "creation_year",
"publication_year",   "restricted",   "license",   "is_readme",   "is_codebook",
"is_data_dictionary", "friendly_format_manual", "is_software", "is_compressed",
"is_microsoft_office"]]

#handles entries where aggregation returned a mixed 'False;True' value
def normalize_boolean_column(col):
```

```
    return col.apply(lambda x: True if isinstance(x, str) and 'true' in x.lower()
else False)
bool_columns = ["is_readme", "is_codebook", "is_data_dictionary", "is_software",
"is_compressed", "is_microsoft_office"]
tdr_all_datasets_deduplicated_pruned                                          =
tdr_all_datasets_deduplicated_pruned.copy()
for col in bool_columns:
                              tdr_all_datasets_deduplicated_pruned[col]       =
normalize_boolean_column(tdr_all_datasets_deduplicated_pruned[col])
tdr_all_datasets_deduplicated_pruned                                          =
tdr_all_datasets_deduplicated_pruned.rename(columns={'is_readme':
'contains_readme',  'is_codebook':  'contains_codebook',  'is_data_dictionary':
'contains_data_dictionary',         'is_software':         'contains_software',
'is_compressed':          'contains_compressed',          'is_microsoft_office':
'contains_microsoft_office', 'file_size': 'dataset_size'})

tdr_all_datasets_deduplicated_pruned['total_version'] = tdr_all_datasets_deduplicated_pruned['tota

#binning datasets by size
tdr_all_datasets_deduplicated_pruned                                          =
assign_size_bins(tdr_all_datasets_deduplicated_pruned,   column='dataset_size',
new_column='dataset_size_bin')

if exclude_drafts:
   tdr_all_datasets_deduplicated_pruned_published = tdr_all_datasets_deduplicated_pruned[tdr_all_d
& (tdr_all_datasets_deduplicated_pruned['publication_date'] != '')]
        tdr_all_datasets_deduplicated_pruned_published.to_csv(f"outputs/{today}
_{institution_filename}_all-datasets-combined-PUBLISHED.csv")
else:
                  tdr_all_datasets_deduplicated_pruned.to_csv(f"outputs/{today}
_{institution_filename}_all-datasets-combined-ALL.csv")
```

Metadata summary

The script also returns two summary files that are likely to be of interest. The first summary is the amount of storage created by calendar year. If these numbers are discordant with previous estimates, there is a very good chance that this is because older versions are not being accounted for (either that files have been replaced or that files existed in the system well before their latest publication). For instance, if you're not paying attention, a versioned dataset most recently published in 2023 but that had a 10 GB file published in 2021, could be mistakenly counted for 2023. The second summary is the number of unique instances of file formats. The way that this calculation is done, it counts each file type once per dataset, so a dataset with 2,000 CSV files and a dataset with 1 CSV file are each counted as '1' for the total number of datasets with CSV files; this is done to eliminate the discipline-specific variability in repetitive file formats (e.g., simulation datasets with millions of text files).

```
#size summary
size_by_year     =     df_all_files_concat_deduplicated.groupby('creation_year')
['file_size'].sum().reset_index()
size_by_year['fileGB'] = size_by_year['file_size'] / 1000000000
print('Annual size summary')
print(size_by_year)
if exclude_drafts:
                                                size_by_year_published           =
df_all_files_concat_deduplicated_published.groupby('creation_year')
['file_size'].sum().reset_index()
     size_by_year_published['fileGB'] = size_by_year_published['file_size'] /
1000000000
         size_by_year_published.to_csv(f"outputs/{today}_{institution_filename}
_SUMMARY-annual-size-PUBLISHED.csv")
else:
          size_by_year.to_csv(f"outputs/{today}_{institution_filename}_SUMMARY-
annual-size-ALL.csv")
```

```
Annual size summary
   creation_year     file_size      fileGB
0           2017  1.528506e+07    0.015285
1           2018  1.349090e+09    1.349090
2           2019  2.931827e+08    0.293183
3           2020  7.414284e+08    0.741428
4           2021  1.183786e+10   11.837860
5           2022  3.446539e+11  344.653947
6           2023  1.172907e+10   11.729066
7           2024  1.021654e+10   10.216540
8           2025  7.743767e+10   77.437669
```

```
#file format summary
##can substitute 'friendly_type' for 'mime_type' but will get some aggregating
into 'unknown'
unique_datasets_per_format                                                    =
df_all_files_concat_deduplicated.groupby('friendly_format_manual')
['dataset_id'].nunique()
print('Total file format summary')
print(unique_datasets_per_format)
if exclude_drafts:
                                    unique_datasets_per_format_PUBLISHED        =
df_all_files_concat_deduplicated_published.groupby('friendly_format_manual')
['dataset_id'].nunique()
                  unique_datasets_per_format_PUBLISHED.to_csv(f"outputs/{today}
_{institution_filename}_SUMMARY-unique-format-PUBLISHED.csv")
else:
```

```
    unique_datasets_per_format.to_csv(f"outputs/{today}_{institution_filename}
_SUMMARY-unique-format-ALL.csv")
```

```
Total file format summary
friendly_format_manual
AVI                            2
Binary                        12
C Source                       1
CSV                           10
DVB Subtitle                   1
Fixed Field                    2
GIF                            1
GZIP (compressed archive)      1
HDF5                           3
JPEG                           2
JSON                           1
Jupyter Notebook               1
KMZ                            1
M4A                            1
MATLAB                         1
MP4                            3
MPEG Audio                     1
MS Excel                       3
MS PowerPoint                  1
MS Shortcut                    1
MS Word                       10
Makefile                       1
Markdown                       4
MiniSEED                       1
NetCDF                         4
PDF                           15
PNG                            8
Pascal                         1
Plain text                     9
PostScript                     1
Python                         4
QuickTime                      1
R Notebook                     3
R Syntax                       5
SVG                            1
TIFF                           9
TSV                           19
ZIP (compressed archive)       5
Zipped Shapefile               2
Name: dataset_id, dtype: int64
```

Metadata assessment: part 4

The last metadata assessment is for the author-level data. The script assesses the following:

- Whether an author has a ROR-linked affiliation
- Whether an author has a properly formatted ORCID (in Dataverse, this means that the ORCID is hyperlinked and that it appears in two fields)
- Whether an author has an ORCID, but it is not properly formatted because it lacks hyphens
- Whether an author has an ORCID, but it is not properly formatted because it is not in URL form
- Whether an author has an ORCID, but it is not properly formatted because it appears in only one of the two fields
- Whether an author has an ORCID, but it is malformed in some fashion (if any of the previous three are TRUE)
- Whether an author's name appears malformed (First Last rather than Last, First); this is based on whether there is a space in the name value but no comma
- Whether an author's initial appears malformed (probably middle, but could be first); this is based on whether there is a standalone letter with no period after it

```
#author assessment
##fuzzy matching author names
unique_names = df_all_authors_concat_deduplicated['author_name'].unique()
standardized_names = {}

for name in unique_names:
# Only try to match if standardized_names is not empty
    if standardized_names:
                result  =  process.extractOne(name,  standardized_names.keys(),
scorer=fuzz.token_sort_ratio)
        if result is not None:
            match, score, _ = result  # rapidfuzz returns (match, score, index)
            if score > 80:  # Adjust threshold as needed
                standardized_names[name] = match
            else:
                standardized_names[name] = name
        else:
            standardized_names[name] = name
    else:
        standardized_names[name] = name  # First name, nothing to match yet
df_all_authors_concat_deduplicated['author_name_standardized']                =
df_all_authors_concat_deduplicated['author_name'].map(standardized_names)

##is ROR present
df_all_authors_concat_deduplicated = df_all_authors_concat_deduplicated.copy()
df_all_authors_concat_deduplicated.loc[:,                          'missing_ror']
=            (df_all_authors_concat_deduplicated['ror_id'].isna()             |
(df_all_authors_concat_deduplicated['ror_id'] == ''))
##is any author ID system present
df_all_authors_concat_deduplicated.loc[:,      'missing_author_scheme']      =
```

```
(df_all_authors_concat_deduplicated['author_identifier_scheme'].isna() |
    (df_all_authors_concat_deduplicated['author_identifier_scheme'] == ''))
##ORCID present and appropriately formatted
df_all_authors_concat_deduplicated.loc[:, 'proper_orcid'] = (
    df_all_authors_concat_deduplicated['author_identifier_scheme'].str.upper()
== 'ORCID'
)                                                                          &
df_all_authors_concat_deduplicated['author_identifier'].str.contains('https://
orcid.org/', na=False)
##is ORCID present but malformatted (not hyperlinked)
df_all_authors_concat_deduplicated.loc[:,'malformed_orcid_no_hyphens'] = (
    df_all_authors_concat_deduplicated['author_identifier_scheme'].str.upper()
== 'ORCID'
) & ~df_all_authors_concat_deduplicated['author_identifier'].str.contains('-',
na=False)
##is ORCID present but malformatted (no dashes)
df_all_authors_concat_deduplicated.loc[:,'malformed_orcid_no_url'] = (
    df_all_authors_concat_deduplicated['author_identifier_scheme'].str.upper()
== 'ORCID'
) & ~df_all_authors_concat_deduplicated['author_identifier'].str.contains('https://
orcid.org/', na=False)
##is ORCID present but malformatted (single field)
df_all_authors_concat_deduplicated.loc[:,'malformed_orcid_single_field'] = (
    df_all_authors_concat_deduplicated['author_identifier_scheme'].str.upper()
== 'ORCID'
) & df_all_authors_concat_deduplicated['author_identifier_expanded'].isna()

df_all_authors_concat_deduplicated.loc[:, 'malformed_orcid_any'] = (
    df_all_authors_concat_deduplicated['malformed_orcid_no_hyphens'] |
    df_all_authors_concat_deduplicated['malformed_orcid_no_url'] |
    df_all_authors_concat_deduplicated['malformed_orcid_single_field']
)
##malformed author name (order)
df_all_authors_concat_deduplicated.loc[:, 'malformed_order'] = (
          df_all_authors_concat_deduplicated['author_name'].str.contains('  ',
na=False) &
           ~df_all_authors_concat_deduplicated['author_name'].str.contains(',',
na=False)
)
##malformed initial (standalone initial without period)
df_all_authors_concat_deduplicated.loc[:,        'malformed_initial']        =
df_all_authors_concat_deduplicated['author_name'].str.contains(r'\b[A-Z]\b(?!
\.)', regex=True)

df_all_authors_concat_deduplicated.loc[:, 'malformed_name'] = (
    df_all_authors_concat_deduplicated['malformed_order'] |
    df_all_authors_concat_deduplicated['malformed_initial']
)
```

```
if exclude_drafts:
   df_all_authors_concat_deduplicated_published = df_all_authors_concat_deduplicated[df_all_author
= 'DRAFT']
          df_all_authors_concat_deduplicated_published.to_csv(f'outputs/{today}
_{institution_filename}_all-authors-PUBLISHED.csv', index=False)
else:
                                    df_all_authors_concat_deduplicated        =
df_all_authors_concat_deduplicated.sort_values(by='author_name')
                    df_all_authors_concat_deduplicated.to_csv(f'outputs/{today}
_{institution_filename}_all-authors-ALL.csv', index=False)

df_all_affiliations_dedup = df_all_authors_concat_deduplicated.drop_duplicates(subset=['author_aff
keep='first')
df_all_affiliations_dedup                                                      =
df_all_affiliations_dedup.rename(columns={'author_affiliation':
'affiliation'})

if master_ror_matching is None: #create master file if it doesn't exist yet
    print("No existing master file found, creating new one.\n")
    print(f"Total unique affiliations: {len(df_all_affiliations_dedup) - 1}\n")
     df_all_affiliations_dedup.to_csv(f'{script_directory}/tdr-affiliation-ror-
matching.csv')
else: #concat master file with new list of unique affiliations, drop duplicates
(keep first will retain existing matches)
    print("Found existing master file, adding and deduplicating.\n")
        df_all_affiliations_dedup_expanded  =  pd.concat([master_ror_matching,
df_all_affiliations_dedup])
    print(f"Total affiliations: {len(df_all_affiliations_dedup_expanded)}\n")
                              df_all_affiliations_dedup_expanded_pruned       =
df_all_affiliations_dedup_expanded.drop_duplicates(subset=['affiliation'],
keep='first')
                                print(f"Total       unique       affiliations:
{len(df_all_affiliations_dedup_expanded_pruned) - 1}\n")
                              df_all_affiliations_dedup_expanded_pruned       =
df_all_affiliations_dedup_expanded_pruned[['affiliation',    'ror',    'flag-
generic']]
    ##remove blanks
                              df_all_affiliations_dedup_expanded_pruned       =
df_all_affiliations_dedup_expanded_pruned.dropna(subset=['affiliation'])
     df_all_affiliations_dedup_expanded_pruned.to_csv(f'{script_directory}/tdr-
affiliation-ror-matching-TEMP.csv')
```

```
Found existing master file, adding and deduplicating.

Total affiliations: 268
```

```
Total unique affiliations: 254
```

Dataverse-level information

This script can also retrieve information on dataverses in TDR. Getting the maximum amount of information requires a similar two-step process to that required for datasets: an initial query through the Search API and then a loop for each dataverse through the Native API (there is no equivalent to the Versions endpoint). The initial set-up repeats many code blocks from the dataset-level retrieval process with minimally modified code - this will likely be consolidated to be more efficient in the future.

```
query = '*'

params_tdr_ut_austin = {
    'q': query,
    'subtree': 'utexas',
    'type': 'dataverse',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_baylor = {
    'q': query,
    'subtree': 'baylor',
    'type': 'dataverse',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_smu = {
    'q': query,
    'subtree': 'smu',
    'type': 'dataverse',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_tamu = {
    'q': query,
    'subtree': 'tamu',
    'type': 'dataverse',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
```

```
}

params_tdr_txst = {
    'q': query,
    'subtree': 'txst',
    'type': 'dataverse',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_ttu = {
    'q': query,
    'subtree': 'ttu',
    'type': 'dataverse',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_houston = {
    'q': query,
    'subtree': 'uh',
    'type': 'dataverse',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_hscfw = {
    'q': query,
    'subtree': 'unthsc',
    'type': 'dataverse',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_tamug = {
    'q': query,
    'subtree': 'tamug',
    'type': 'dataverse',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}
params_tdr_tamui = {
    'q': query,
```

```
    'subtree': 'tamiu',
    'type': 'dataverse',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}
params_tdr_utsah = {
    'q': query,
    'subtree': 'uthscsa',
    'type': 'dataverse',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}
params_tdr_utswm = {
    'q': query,
    'subtree': 'utswmed',
    'type': 'dataverse',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_uta = {
    'q': query,
    'subtree': 'uta',
    'type': 'dataverse',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

params_tdr_twu = {
    'q': query,
    'subtree': 'twu',
    'type': 'dataverse',
    'start': page_start_dataverse,
    'page': page_increment,
    'per_page': page_limit_dataverse
}

all_params = {
        "UT Austin": params_tdr_ut_austin,
        "Baylor": params_tdr_baylor,
        "SMU": params_tdr_smu,
        "TAMU": params_tdr_tamu,
        "Texas State": params_tdr_txst,
        "Texas Tech": params_tdr_ttu,
```

```
        "Houston": params_tdr_houston,
        "HSC Fort Worth": params_tdr_hscfw,
        "TAMU Galveston": params_tdr_tamug,
        "TAMU International": params_tdr_tamui,
        "UT San Antonio Health": params_tdr_utsah,
        "UT Southwestern Medical": params_tdr_utswm,
        "UT Arlington": params_tdr_uta,
        "Texas Women's University": params_tdr_twu
    }

tamu_combined_params = {
        "TAMU": params_tdr_tamu,
        "TAMU Galveston": params_tdr_tamug,
        "TAMU International": params_tdr_tamui
}

#substitute for your institution
if only_my_institution:
    if my_institution_shortName == "TAMU":
        params_list = tamu_combined_params
    else:
        params_list = {
            my_institution_shortName: all_params[my_institution_shortName]
        }
else:
    params_list = all_params

print("Beginning to define API call parameters.")

##(re)set API-specific params
page_limit_dataverse = config['VARIABLES']['PAGE_LIMITS']['tdr_test'] if test
else config['VARIABLES']['PAGE_LIMITS']['tdr_prod']
page_size  =  config['VARIABLES']['PAGE_SIZES']['dataverse']  if  test  else
config['VARIABLES']['PAGE_SIZES']['dataverse']
print(f"Retrieving {page_size} records per page over {page_limit_dataverse}
pages.")

###for TDR, affiliation is not reliable for returning all relevant results; the
DOI prefix is used as the most generic common denominator for datasets
query = '10.18738/T8/'
page_start_dataverse = config['VARIABLES']['PAGE_STARTS']['dataverse']
page_increment = config['VARIABLES']['PAGE_INCREMENTS']['dataverse']
k = 0

print("Starting TDR retrieval.\n")
all_dataverses  =  retrieve_all_data_for_institutions(url_tdr,  params_list,
headers_tdr)
```

```
print("Starting TDR filtering.\n")
dataverses_select_tdr = []
for item in all_dataverses:
    name = item.get('name', '')
    type = item.get('type', '')
    url = item.get('url','')
    identifier = item.get('identifier', '')
    description = item.get('description', '')
    published = item.get('published_at', '')
    status = item.get('publicationStatuses', '')
    affiliation = item.get('affiliation', '')
    parent_dataverse_name = item.get('parentDataverseName', '')
    parent_dataverse_id = item.get('parentDataverseIdentifier', '')
    dataverses_select_tdr.append({
        'dataverse_name': name,
        'url': url,
        'identifier': identifier,
        'type': type,
        'status': status,
        'description': description,
        'published': published,
        'affiliation': affiliation,
        'parent_dataverse_name': parent_dataverse_name,
        'parent_dataverse_id': parent_dataverse_id
    })

df_dataverses_select_tdr = pd.DataFrame(dataverses_select_tdr)
```

```
Beginning to define API call parameters.
Retrieving 200 records per page over 10 pages.
Starting TDR retrieval.

Retrieving Page 1 of 44 pages...

Retrieving Page 2 of 44 pages...

Retrieving Page 3 of 44 pages...

Retrieving Page 4 of 44 pages...

Retrieving Page 5 of 44 pages...

Retrieving Page 6 of 44 pages...

Retrieving Page 7 of 44 pages...

Retrieving Page 8 of 44 pages...
```

```
Retrieving Page 9 of 44 pages...

Retrieving Page 10 of 44 pages...

Starting TDR filtering.
```

Now we loop the '*identifier*' field through the Native API:

```
print("Starting Native API call")
url_tdr_native = "https://dataverse.tdl.org/api/dataverses/"

results = []
for identifier in df_dataverses_select_tdr['identifier']:
    try:
                    response  =  requests.get(f'{url_tdr_native}/{identifier}',
headers=headers_tdr, timeout=5)
        if response.status_code == 200:
            print(f"Retrieving {identifier}\n")
            results.append(response.json())
        else:
                      print(f"Error  retrieving  {doi}:  {response.status_code},
{response.text}")
    except requests.exceptions.RequestException as e:
        print(f"Timeout error on dataverse {identifier}: {e}")

data_tdr_native = {
    'dataverses': results
}

print("Beginning dataframe subsetting\n")
data_select_tdr_native = []
for item in data_tdr_native['dataverses']:
    data = item.get('data')
    id = data.get('id', '')
    contacts = data.get('dataverseContacts', [])
    contact_email = [contact.get('contactEmail', None) for contact in contacts
if isinstance(contact, dict)]
    contactsCount = len(contact_email)
    permission = data.get('permissionRoot', '')
    dataverse_type = data.get('dataverseType', '')
    metadata_block = data.get('isMetadataBlockRoot', '')
    facet_root = data.get('isFacetRoot', '')
    owner = data.get('ownerId', '')
    created = data.get('creationDate', '')
    identifier = data.get('alias', '')
    released = data.get('isReleased', '')
```

```
    data_select_tdr_native.append({
        'id': id,
        'contact_email': contact_email,
        'contact_count': contactsCount,
        'owner': owner,
        'dataverse_type': dataverse_type,
        'permission': permission,
        'metadata_block': metadata_block,
        'facet_root': facet_root,
        'created': created,
        'identifier': identifier,
        'released': released
    })

df_select_tdr_native = pd.json_normalize(data_select_tdr_native)

#merge dfs
df_select_concatenated          =          pd.merge(df_dataverses_select_tdr,
df_select_tdr_native, on='identifier', how="left")
```

```
Starting Native API call
Retrieving orthographiceeg

Retrieving Yu_Hodges_Liu_2019

Retrieving multifactorial_prediction_depression

Retrieving lomax_savvaidis_2019

Retrieving ReferenceSlope

Retrieving dougNVF

Retrieving RAnalysisVis

Retrieving bridging-barriers

Retrieving tianyisun

Retrieving pt2050

Retrieving good-systems

Retrieving whole-communities

Retrieving burkholderia
```

```
Retrieving PamelaSpeciale

Retrieving PaleoBoardGames

Retrieving Taphonomy_Dead_and_Fossilized

Retrieving OilSpill

Retrieving CETCS_ExVivoPorcine

Retrieving Du_Mehmani_et_al_WRR_2020

Retrieving Jezero_Delta_3D_Print

Retrieving skung

Retrieving UTFT

Retrieving beccs_neg_emission

Retrieving texnet-cisr

Retrieving texnet-cisr-dfw

Retrieving texnet-cisr-permian

Retrieving Devils_River_TX

Retrieving hnr393

Retrieving joanehughes

Retrieving cogbias_reward

Retrieving Texas_Earthquake_20200326

Retrieving FishCamouflage

Retrieving TDLforMAs

Retrieving arts_partnership_network

Retrieving kraley

Retrieving childhh

Retrieving HydrocarbonProduction
```

```
Retrieving CDC

Retrieving edtechnet

Retrieving LatticeModelFracture

Retrieving sgani

Retrieving frehdc

Retrieving Schorghofer_RSL_DEMs

Retrieving iel

Retrieving acetaminophen

Retrieving 3dem

Retrieving constitutionalreferenda

Retrieving utlarch

Retrieving bot

Retrieving abm

Retrieving joshconrad-dissertation

Retrieving lanic

Retrieving PRC

Retrieving hodges

Retrieving SaintVenant

Retrieving SvePy

Retrieving SvePy_output_flow_over_bump

Retrieving SvePy_output_Waller_Creek

Retrieving SvePy_source_code

Retrieving gaia

Retrieving UTCSR
```

```
Retrieving schmandt-besserat

Retrieving utblac_lanic_castrospeeches

Retrieving cog_bias_sym

Retrieving CDBNgenomics

Retrieving pantanello

Retrieving croton

Retrieving sav

Retrieving negsr5httlpr

Retrieving utblac_rgs

Retrieving HRC

Retrieving lhr

Retrieving jacob

Retrieving spncrfx

Retrieving Rocks

Retrieving Bio

Retrieving Paleo

Retrieving Other

Retrieving chatino

Retrieving keitz

Retrieving pasp

Retrieving admin

Retrieving chersonesos

Retrieving marcosperezdataverse

Retrieving stel
```

```
Retrieving blac_rare_books_maps

Retrieving genarogarcia

Retrieving testMiranker

Retrieving gbm_epigenetics

Retrieving psrt

Retrieving morphodynamics

Retrieving ctr-transportation

Retrieving transportation-library

Retrieving mrgardner

Retrieving wilkelab

Retrieving llilasbenson

Retrieving BernardBehr2017

Retrieving NARLS

Retrieving mdl

Retrieving sretdepression

Beginning dataframe subsetting
```

```
url_contents = "https://dataverse.tdl.org/api/dataverses/{}/contents"
url_storagesize = "https://dataverse.tdl.org/api/dataverses/{}/storagesize"

contents_results = []
storagesize_results = []

for identifier in df_dataverses_select_tdr['identifier']:
    # Fetch contents
    try:
                    response  =  requests.get(url_contents.format(identifier),
headers=headers_tdr, timeout=5)
        if response.status_code == 200:
            print(f"Retrieving contents for {identifier}\n")
            contents_results.append(response.json())
        else:
```

```
                    print(f"Error retrieving contents for {identifier}:
{response.status_code}, {response.text}\n")
            contents_results.append(None)
    except requests.exceptions.RequestException as e:
        print(f"Timeout error on contents {identifier}: {e}\n")
        contents_results.append(None)

    # # Fetch storagesize
    # try:
       #         response = requests.get(url_storagesize.format(identifier),
headers=headers_tdr, timeout=5)
    #     if response.status_code == 200:
    #         storagesize_results.append(response.json())
    #     else:
      #             print(f"Error retrieving storagesize for {identifier}:
{response.status_code}, {response.text}")
    #         storagesize_results.append(None)
    # except requests.exceptions.RequestException as e:
    #     print(f"Timeout error on storagesize {identifier}: {e}")
    #     storagesize_results.append(None)

# df_select_concatenated['contents'] = contents_results
# df_select_concatenated['storagesize'] = storagesize_results

#reporting number of datasets and dataverses
# Count dataverses
num_dataverses = [
    sum(1 for item in (contents.get('data', []) if isinstance(contents, dict)
else [])
        if item.get('type') == 'dataverse')
    for contents in contents_results
]

# Count datasets
num_datasets = [
    sum(1 for item in (contents.get('data', []) if isinstance(contents, dict)
else [])
        if item.get('type') == 'dataset')
    for contents in contents_results
]

# Get dataset DOIs
dataset_dois = [
    '; '.join(
        item.get('persistentUrl', '').replace('https://doi.org/', '')
       for item in (contents.get('data', []) if isinstance(contents, dict) else
[])
        if item.get('type') == 'dataset' and 'persistentUrl' in item
```

```
    )
    for contents in contents_results
]

# Add to DataFrame
df_select_concatenated['num_dataverses'] = num_dataverses
df_select_concatenated['num_datasets'] = num_datasets
df_select_concatenated['dataset_dois'] = dataset_dois

df_select_concatenated.to_csv(f"outputs/{today}_{institution_filename}_all-
dataverses.csv")
df_select_concatenated_pruned    =    df_select_concatenated[['dataverse_name',
'url',  'identifier',  'parent_dataverse_name',  'parent_dataverse_id',  'id',
'contact_email', 'owner', 'dataset_dois']]

if exclude_drafts:
    dataverse_dataset_merged = pd.merge(
        tdr_all_datasets_deduplicated_pruned_published,
        df_select_concatenated_pruned,
        left_on='dataverse',
        right_on='dataverse_name',
        how='left'
    )
                                          dataverse_dataset_merged          =
dataverse_dataset_merged.fillna({'dataverse_name':    'Default    institutional
dataverse',   'parent_dataverse_name':   'None',   'parent_dataverse_id':   0,
'contact_email': 'None', 'owner': 0, 'id': 0, 'dataset_dois': 'Not applicable'})

      dataverse_dataset_merged.to_csv(f"outputs/{today}_{institution_filename}
_all-datasets-combined-with-dataverses-PUBLISHED.csv")
else:
    dataverse_dataset_merged = pd.merge(
        tdr_all_datasets_deduplicated_pruned,
        df_select_concatenated_pruned,
        left_on='dataverse',
        right_on='dataverse_name',
        how='left'
    )
                                          dataverse_dataset_merged          =
dataverse_dataset_merged.fillna({'dataverse_name':    'Default    institutional
dataverse',   'parent_dataverse_name':   'None',   'parent_dataverse_id':   0,
'contact_email': 'None', 'owner': 0, 'id': 0, 'dataset_dois': 'Not applicable'})

      dataverse_dataset_merged.to_csv(f"outputs/{today}_{institution_filename}
_all-datasets-combined-with-dataverses-ALL.csv")
```

```
Retrieving contents for orthographiceeg
```

```
Retrieving contents for Yu_Hodges_Liu_2019

Retrieving contents for multifactorial_prediction_depression

Retrieving contents for lomax_savvaidis_2019

Retrieving contents for ReferenceSlope

Retrieving contents for dougNVF

Retrieving contents for RAnalysisVis

Retrieving contents for bridging-barriers

Retrieving contents for tianyisun

Retrieving contents for pt2050

Retrieving contents for good-systems

Retrieving contents for whole-communities

Retrieving contents for burkholderia

Retrieving contents for PamelaSpeciale

Retrieving contents for PaleoBoardGames

Retrieving contents for Taphonomy_Dead_and_Fossilized

Retrieving contents for OilSpill

Retrieving contents for CETCS_ExVivoPorcine

Retrieving contents for Du_Mehmani_et_al_WRR_2020

Retrieving contents for Jezero_Delta_3D_Print

Retrieving contents for skung

Retrieving contents for UTFT

Retrieving contents for beccs_neg_emission

Retrieving contents for texnet-cisr

Retrieving contents for texnet-cisr-dfw
```

```
Retrieving contents for texnet-cisr-permian

Retrieving contents for Devils_River_TX

Retrieving contents for hnr393

Retrieving contents for joanehughes

Retrieving contents for cogbias_reward

Retrieving contents for Texas_Earthquake_20200326

Retrieving contents for FishCamouflage

Retrieving contents for TDLforMAs

Retrieving contents for arts_partnership_network

Retrieving contents for kraley

Retrieving contents for childhh

Retrieving contents for HydrocarbonProduction

Retrieving contents for CDC

Retrieving contents for edtechnet

Retrieving contents for LatticeModelFracture

Retrieving contents for sgani

Retrieving contents for frehdc

Retrieving contents for Schorghofer_RSL_DEMs

Retrieving contents for iel

Retrieving contents for acetaminophen

Retrieving contents for 3dem

Retrieving contents for constitutionalreferenda

Retrieving contents for utlarch

Retrieving contents for bot
```

```
Retrieving contents for abm
Retrieving contents for joshconrad-dissertation
Retrieving contents for lanic
Retrieving contents for PRC
Retrieving contents for hodges
Retrieving contents for SaintVenant
Retrieving contents for SvePy
Retrieving contents for SvePy_output_flow_over_bump
Retrieving contents for SvePy_output_Waller_Creek
Retrieving contents for SvePy_source_code
Retrieving contents for gaia
Retrieving contents for UTCSR
Retrieving contents for schmandt-besserat
Retrieving contents for utblac_lanic_castrospeeches
Retrieving contents for cog_bias_sym
Retrieving contents for CDBNgenomics
Retrieving contents for pantanello
Retrieving contents for croton
Retrieving contents for sav
Retrieving contents for negsr5httlpr
Retrieving contents for utblac_rgs
Retrieving contents for HRC
Retrieving contents for lhr
Retrieving contents for jacob
```

```
Retrieving contents for spncrfx

Retrieving contents for Rocks

Retrieving contents for Bio

Retrieving contents for Paleo

Retrieving contents for Other

Retrieving contents for chatino

Retrieving contents for keitz

Retrieving contents for pasp

Retrieving contents for admin

Retrieving contents for chersonesos

Retrieving contents for marcosperezdataverse

Retrieving contents for stel

Retrieving contents for blac_rare_books_maps

Retrieving contents for genarogarcia

Retrieving contents for testMiranker

Retrieving contents for gbm_epigenetics

Retrieving contents for psrt

Retrieving contents for morphodynamics

Retrieving contents for ctr-transportation

Retrieving contents for transportation-library

Retrieving contents for mrgardner

Retrieving contents for wilkelab

Retrieving contents for llilasbenson

Retrieving contents for BernardBehr2017
```

```
Retrieving contents for NARLS

Retrieving contents for mdl

Retrieving contents for sretdepression
```

```
print("Done.\n")
if master_ror_matching is not None:
                  print(f"Number    of    new    affiliations    to    check:
{len(df_all_affiliations_dedup_expanded_pruned)  -  len(master_ror_matching)}.
\n")
print(f"Time to run: {datetime.now() - start_time}\n")
if test:
   print("**REMINDER: THIS IS A TEST RUN, AND ANY RESULTS ARE NOT COMPLETE!**\n")
```

```
Done.

Number of new affiliations to check: 0.

Time to run: 0:01:50.314956

**REMINDER: THIS IS A TEST RUN, AND ANY RESULTS ARE NOT COMPLETE!**
```